SEV-2

SEV-2: IAM Access Denied

A post-mortem playbook for hunting down and surgically fixing AWS cloud permission failures.

AWS Day 2 Operations & Troubleshooting · NorthBank

NotebookLM

The Alert Widget

Status: ACTIVE

Reporter: Marco (App Team)

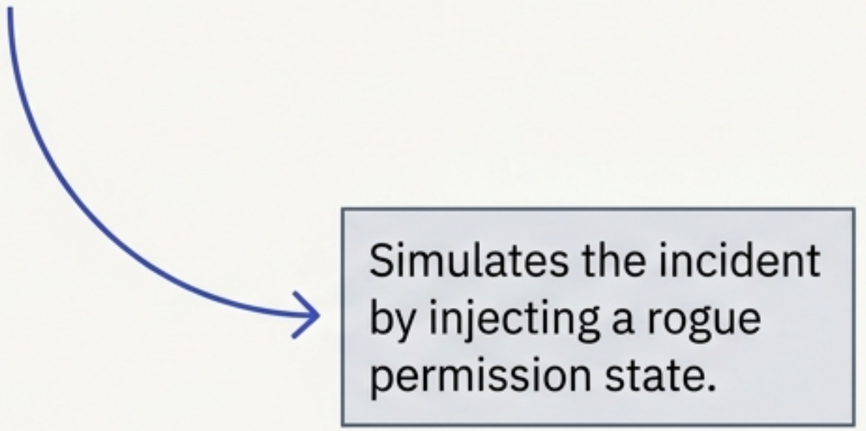
System: NorthBank Production

Our app suddenly gets Access Denied writing to its S3 bucket. We didn't change anything!

The Environment State

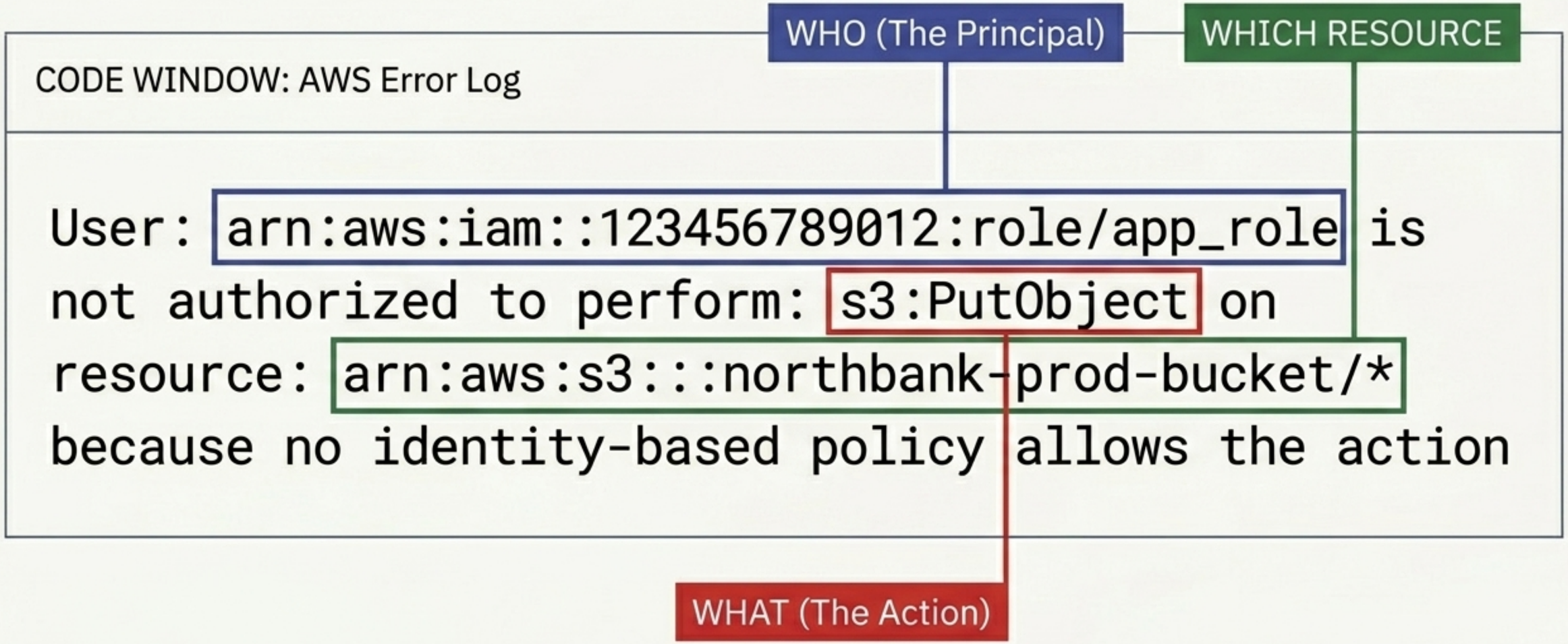
TERMINAL: PRODUCTION ENVIRONMENT

```
$ export ROLE_NAME=$(terraform output
-raw app_role)
$ ./break.sh
```



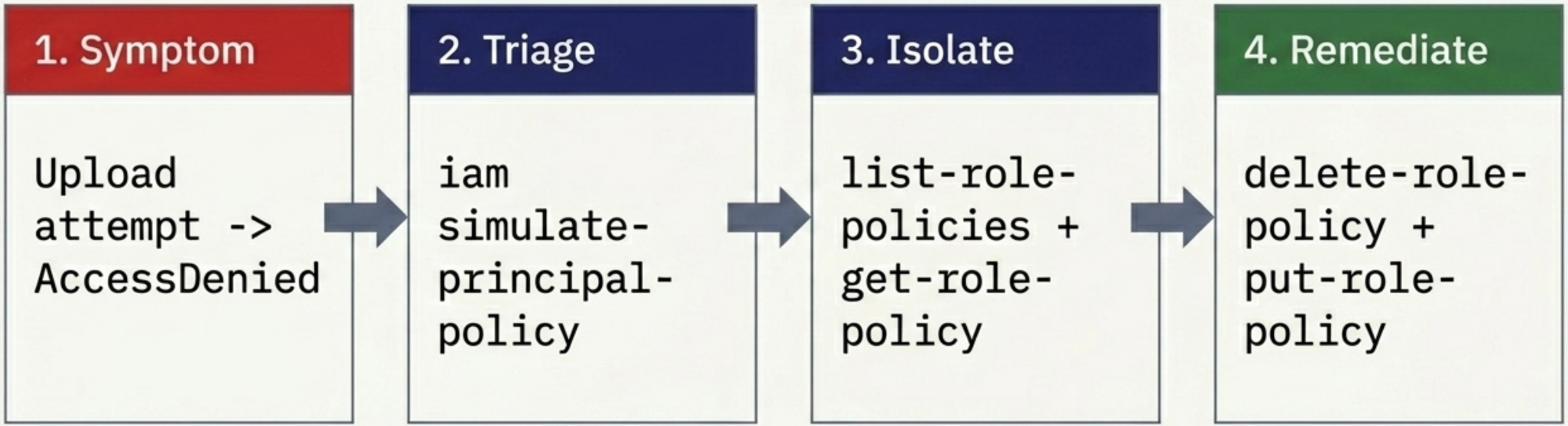
NotebookLM

Never guess. An AccessDenied log always contains the exact coordinates of the failure. Parse these three variables before touching the console.



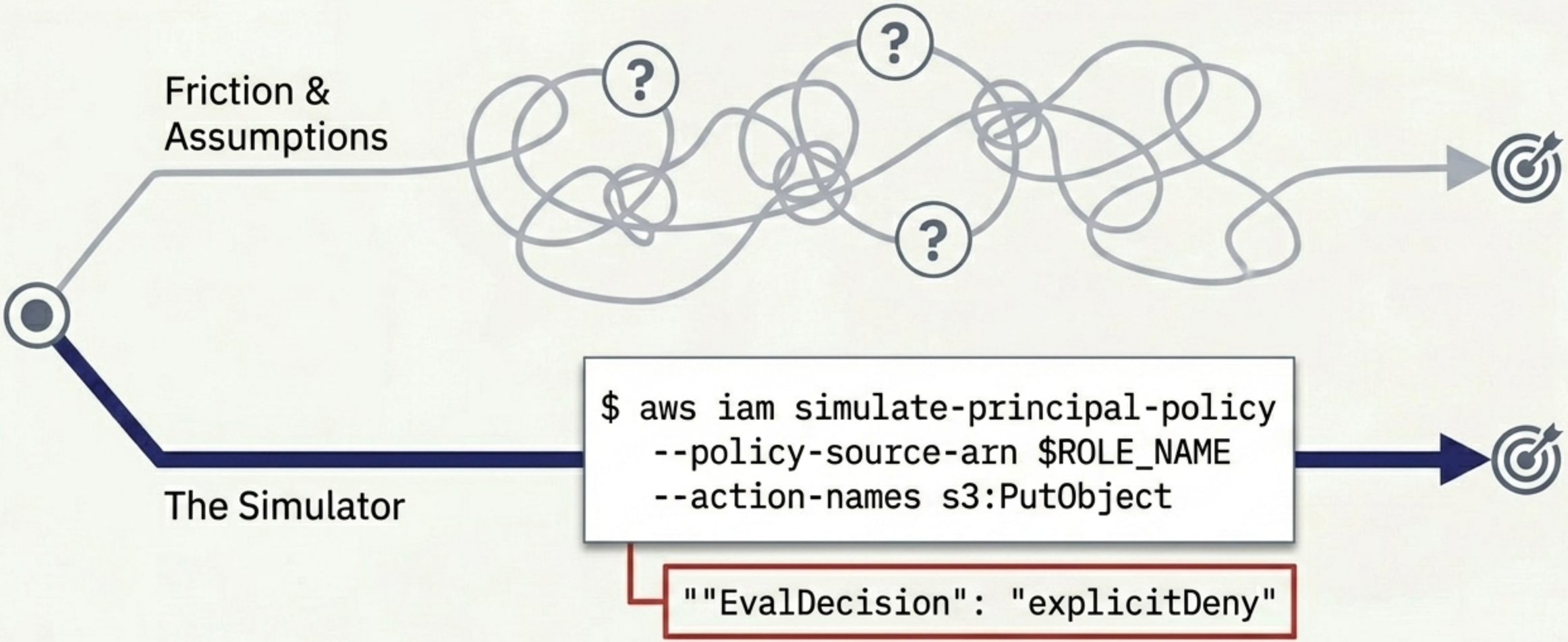
NotebookLM

The Diagnostic Pipeline. Move systematically from the symptom to the exact JSON configuration without relying on trial and error.



NotebookLM

Prove it, don’t guess. The IAM simulator cuts through the noise and directly queries the evaluation engine to confirm the exact policy decision.



NotebookLM

List Role Policies

```
$ aws iam list-role-policies --role-name
$ROLE_NAME
```

```
{
  "PolicyNames": [
    "read-only-access",
    "standard-s3-access",
    "rogue-deny-policy",
    "ec2-management"
  ]
}
```

Get Role Policy

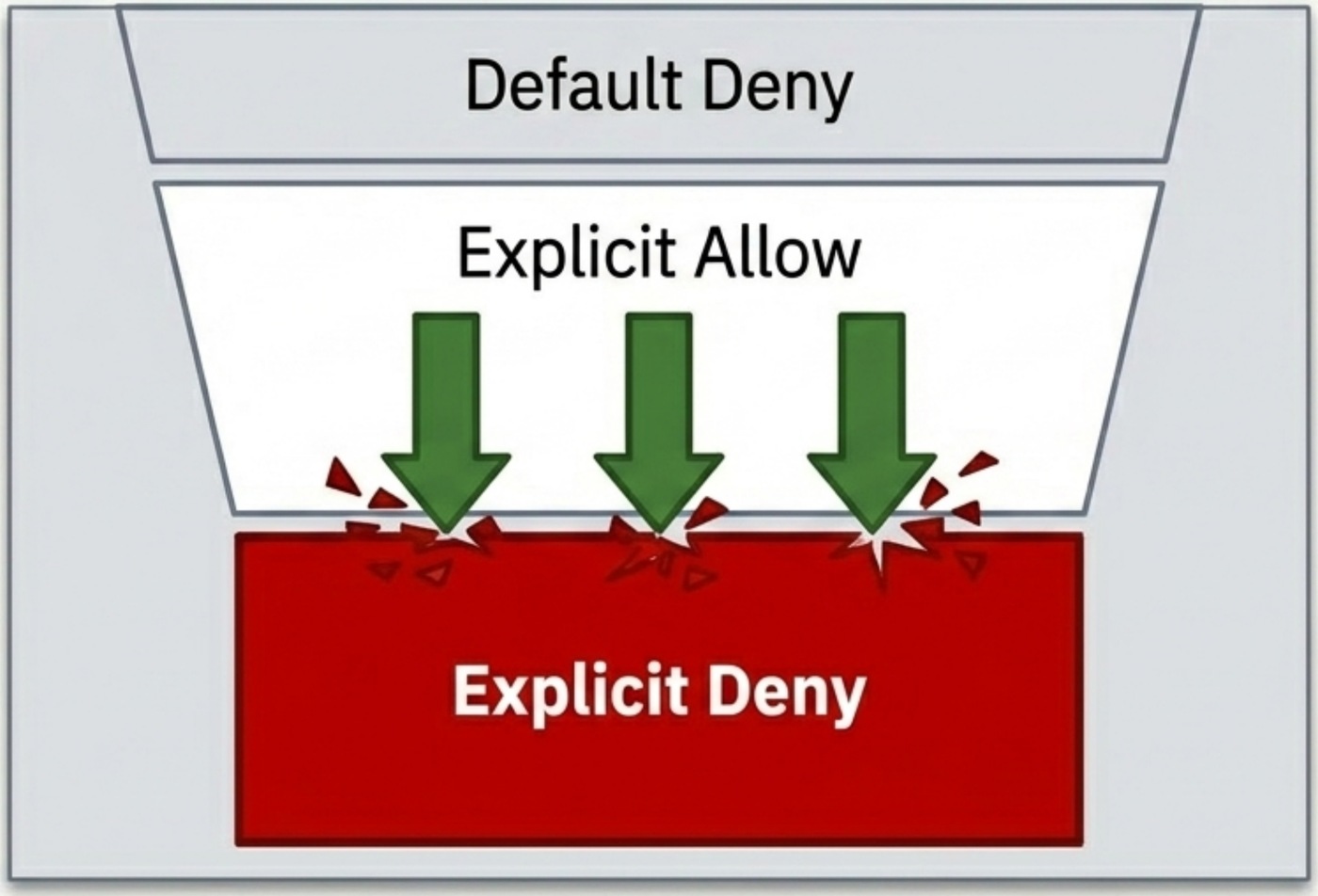
```
$ aws iam get-role-policy --role-name
$ROLE_NAME --policy-name rogue-deny-policy
```

```
{
  "RoleName": "$ROLE_NAME",
  "PolicyName": "rogue-deny-policy",
  "PolicyDocument": {
    "Version": "2012-10-17",
    "Statement": [
      {
        "Sid": "DenyS3Put",
        "Effect": "Deny",
        "Action": "s3:PutObject",
        "Resource": "arn:aws:s3:::my-critical-bucket/*"
      }
    ]
  }
}
```

Isolating the offender. Once the simulator proves a deny exists, list the attachments and retrieve the specific JSON document.

NotebookLM

An Explicit Deny always wins. It operates as an impenetrable firewall at the top of the evaluation logic, instantly crushing any underlying ‘Allow’ rules. Hunt for it first.



NotebookLM

Fix with the exact action needed, not a blanket allow.

The Remediation Matrix

The Sledgehammer

```
"Action": "s3:*"
```

Speed to Resolve: **Fast**

Security Posture: **Poor**

Blast Radius: **High**

Audit Compliance: **Fails**

The Scalpel

```
"Action": "s3:PutObject"
```

Speed to Resolve: **Slightly slower**

Security Posture: **Excellent**

Blast Radius: **Zero**

Audit Compliance: **Passes**

NotebookLM

Remove the Bad

```
$ aws iam delete-role-policy
  --role-name $ROLE_NAME
  --policy-name rogue-deny-policy
```

Inject the Precise Rule

```
$ aws iam put-role-policy
  --role-name $ROLE_NAME
  --policy-document
 file://precise-allow.json
```

```
{
  "Action": "s3:PutObject"
  "Resource": "..."
}
```

The Verification

```
$ aws iam simulate-principal-policy
  --policy-source-arn
arn:aws:iam::123456789012:role/$ROLE_NAME
  --action-names s3:PutObject
```

```
"EvalDecision": "allowed"
```

Surgical Remediation. Remove the explicit deny and replace it with a least-privilege explicit allow. Always verify the fix with the simulator before notifying the app team.

NotebookLM

Explicit Deny beats everything.

When unexpected Access Denied errors occur, always hunt for the Explicit Deny first.

Today’s shortcut is next month’s finding.

Fix with a scalpel, never a sledgehammer. Restrict policies to the exact action required.

Incident Closed. NorthBank System Restored.

NotebookLM